

# MazeGenerator — dokument teoretyczny

Opis działania programu, struktur danych oraz algorytmów generowania i rozwiązywania labiryntu.

## 1. Ogólna idea programu

Program tworzy labirynt jako siatkę komórek. Każda komórka zna swoje współrzędne oraz zbiór połączeń z innymi komórkami. Jeżeli dwie sąsiednie komórki są połączone, oznacza to, że między nimi nie ma ściany i można między nimi przejść.

Po wygenerowaniu labiryntu program traktuje komórki jako wierzchołki grafu, a połączenia między komórkami jako krawędzie grafu. Następnie algorytmy A\* i Dijkstra szukają najkrótszej ścieżki od komórki startowej do końcowej.

## 2. Model danych

### Cell

Komórka przechowuje współrzędne x i y oraz zbiór połączeń z innymi komórkami. Zbiór Set zapobiega duplikatom połączeń.

### Wall

Ściana reprezentuje parę sąsiednich komórek. W czasie generowania Kruskal decyduje, czy ścianę należy usunąć, czyli czy komórki należy połączyć.

### Maze

Abstrakcyjna klasa Maze przechowuje szerokość, wysokość oraz dwuwymiarową siatkę komórek. Udostępnia też metody do inicjalizacji siatki, budowy listy ścian i tasowania Fishera-Yatesa.

## 3. Reprezentacja labiryntu jako grafu

Labirynt jest grafem nieskierowanym:

- wierzchołek grafu = jedna komórka labiryntu,
- krawędź grafu = przejście między dwiema sąsiednimi komórkami,
- brak krawędzi = ściana między komórkami.

Metoda `connections()` w klasie `Cell` zwraca dostępnych sąsiadów. Solvery nie muszą znać ścian bezpośrednio; wystarczy, że iterują po połączeniach bieżącej komórki.

## 4. Disjoint-Set / Union-Find

`DisjointSet` służy do sprawdzania, czy dwie komórki należą już do tej samej składowej. Jest to kluczowe w algorytmie Kruskala, ponieważ usunięcie ściany między komórkami już połączonymi utworzyłoby cykl.

Struktura używa dwóch optymalizacji:

- kompresja ścieżki w `find(x)`, która skraca drogę od elementu do reprezentanta zbioru,
- łączenie według rangi w `union(a, b)`, które ogranicza wysokość drzew reprezentujących zbiory.

## 5. Generowanie labiryntu — Kruskal

Zrandomizowany algorytm Kruskala generuje labirynt przez wybieranie ścian w losowej kolejności i usuwanie tylko tych, które łączą różne zbiory komórek.

Kroki algorytmu:

- utworzenie siatki komórek,
- utworzenie listy ścian między sąsiednimi komórkami,
- przetasowanie ścian algorytmem Fishera-Yatesa,
- utworzenie osobnego zbioru dla każdej komórki,
- iteracja po ścianach i usuwanie tych, które łączą różne zbiory,
- łączenie zbiorów po usunięciu ściany.

W efekcie powstaje perfect maze: labirynt spójny i bez cykli. Między dowolnymi dwiema komórkami istnieje dokładnie jedna ścieżka.

## 6. Fisher-Yates

Fisher-Yates to algorytm równomiernego tasowania tablicy. W projekcie tasowana jest lista wszystkich ścian przed główną pętlą Kruskala. Dzięki temu kolejność rozpatrywania ścian jest losowa, a każdy wygenerowany labirynt może wyglądać inaczej.

```
dla i od arr.length - 1 do 1:  
  j = losowa liczba całkowita z zakresu [0, i]  
  zamień arr[i] z arr[j]
```

## 7. Rozwiązywanie labiryntu — Dijkstra

Algorytm Dijkstry znajduje najkrótszą ścieżkę w grafie z nieujemnymi wagami. W tym projekcie każde przejście między komórkami ma koszt 1.

Dijkstra przechowuje odległość od startu do każdej komórki oraz poprzednika, z którego przyszło najlepsze dojście. Używa kolejki priorytetowej MinHeap, aby zawsze przetwarzać komórkę o najmniejszym znanym koszcie.

Dijkstra nie używa heurystyki, dlatego często odwiedza więcej komórek niż A\*.

## 8. Rozwiązywanie labiryntu — A\*

A\* działa podobnie do Dijkstry, ale dodaje heurystykę, czyli oszacowanie odległości od bieżącej komórki do celu. W projekcie używana jest heurystyka Manhattan:

$$h(a, b) = |a.x - b.x| + |a.y - b.y|$$

A\* wybiera komórkę o najmniejszej wartości fScore:

$$fScore = gScore + hScore$$

- gScore — koszt dojścia od startu do bieżącej komórki,
- hScore — heurystyczny koszt dojścia od bieżącej komórki do celu,
- fScore — suma obu wartości.

## 9. Dlaczego A\* i Dijkstra zwracają tę samą długość ścieżki?

Ponieważ labirynt generowany Kruskalem jest perfect maze, między startem i końcem istnieje dokładnie jedna ścieżka. Z tego powodu oba algorytmy powinny znaleźć ścieżkę o tej samej długości. Różnica widoczna jest głównie w liczbie odwiedzonych komórek oraz w sposobie rozchodzenia się animacji.

Gdyby labirynt zawierał cykle i kilka równoważnych najkrótszych tras, A\* i Dijkstra mogłyby zwrócić różne ścieżki, ale nadal o minimalnej długości.

## 10. Animacja działania algorytmów

Solvery zapisują kolejność odwiedzania komórek w tablicy visitedOrder. Frontend wykorzystuje tę tablicę do animacji. Najpierw rysowane są odwiedzone komórki, a po zakończeniu wyszukiwania rysowana jest finalna ścieżka.

- kolor żółty oznacza komórki odwiedzone podczas wyszukiwania,
- kolor pomarańczowy oznacza znalezioną ścieżkę,
- kolor zielony oznacza start,
- kolor czerwony oznacza koniec.

## 11. Pobieranie PNG

Opcja pobierania PNG tworzy pomocniczy canvas, kopiuje na niego widoki A\* i Dijkstry, dodaje podpisy, a następnie tworzy link z danymi obrazu przez `toDataURL('image/png')`. Kliknięcie linku uruchamia pobieranie pliku.

## 12. Podsumowanie

Program łączy elementy programowania obiektowego, struktur danych, algorytmów grafowych i wizualizacji. Najważniejszy przepływ danych wygląda następująco:

```
Cell + Wall → Maze → KruskalMaze → graf po■czcie■ → Solver → CanvasRenderer → animacja w przegl■darce
```

Najważniejsza obserwacja

Kruskal tworzy labirynt bez cykli, dlatego ścieżka między startem a końcem jest unikalna. A\* i Dijkstra dochodzą więc do tej samej trasy, ale mogą odwiedzić różną liczbę komórek.